

Objectifs : Ce TD/TP illustre la partie du cours sur le développement d'application avec des interfaces humain-machine. Le concept principal abordé est le développement d'une interface simple en html/css/javascript.

1 Description d'une application javascript: décompter les votes, les fonctionnalités attendues

Les figures suivantes montrent la page finale à différente phase. Nous nous plaçons dans le cadre d'un vote par « adhésion », où chaque votant choisi un nombre fixé de candidats. Ici, c'est fixé à 5, chaque bulletin doit avoir exactement 5 candidats différents, sinon il est nul. L'objectif est de « classer » les candidats en fonction du nombre de voix.



Figure 1. L'état initial (affichage en « lignes »)



Figure 2. Pendant la saisie d'un vote

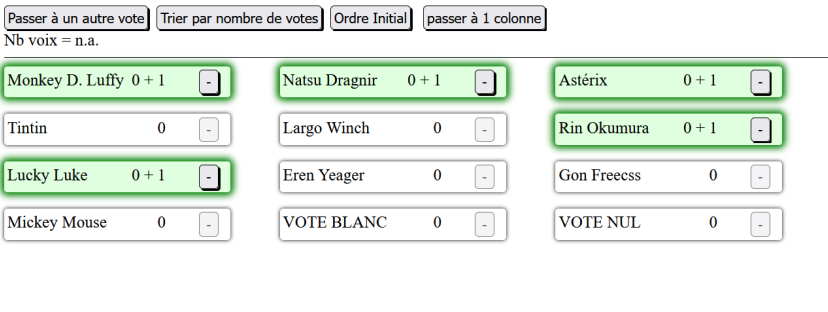


Figure 3. le bulletin est entièrement saisi

La Figure 1 montre la page à l'état initial : les « candidats » sont affichés en ligne, ils n'ont aucune voix. Certains boutons sont désactivés.

L'affichage en ligne permet d'avoir une vue d'ensemble plus pratique pour la saisie ou pour suivre l'évolution.

Comme l'illustre la Figure 2, pendant qu'on saisit un bulletin de vote, les candidats recevant un vote sont mis en valeur et un affichage temporaire permet de suivre l'évolution du nombre de voix.

« Vote blanc » ou « Vote nul » sont exclusifs avec le vote des candidats (et entre eux).

Il faut 5 « voix » ou un « Vote blanc » ou « Vote nul » pour pouvoir passer à un autre bulletin de vote.

Quand un bulletin est entièrement saisi (5 voix comme sur la Figure 3 ou un « Vote blanc » ou « Vote nul ») alors il n'est plus possible d'ajouter une voix et on peut passer à un autre bulletin de vote. Il y a donc un contrôle sur la validité d'un bulletin.

Quand un candidat (ou « Vote blanc » ou « Vote nul ») est sélectionné, le bouton « - » (moins) associé est activé pour pouvoir corriger une erreur de sélection.



Passer à un autre vote | Trier par nombre de votes | Ordre Initial | passer à 1 colonne

Nb voix = 25 (nb votants= 5.00)

Monkey D. Luffy	3	-	Natsu Dragnir	3	-	Astérix	3	-
Tintin	0	-	Largo Winch	2	-	Rin Okumura	1	-
Lucky Luke	1	-	Eren Yeager	2	-	Gon Freecss	0	-
Mickey Mouse	0	-	VOTE BLANC	1	-	VOTE NUL	1	-

Figure 4. Au début de la saisie d'un nouveau bulletin après plusieurs autres saisies

Après la saisie d'un bulletin, à la différence du nombre de voix, on revient à l'état proche de l'état initial.

Au fur et à mesure que les bulletins sont saisis, le nombre de voix est affichée (un bulletin blanc ou nul compte pour 5). Sur la Figure 4, il y a eu 3 bulletins complets, un bulletin nul et un bulletin blanc. Le nombre de voix (un multiple de 5) ou le nombre de votant (un nombre entier) est une indication visuelle du bon déroulement du procédé.

Passer à un autre vote | Trier par nombre de votes | Ordre Initial | passer à 1 colonne

Nb voix = 25 (nb votants= 5.00)

Monkey D. Luffy	3	-	Natsu Dragnir	3	-	Astérix	3	-
Largo Winch	2	-	Eren Yeager	2	-	Rin Okumura	1	-
Lucky Luke	1	-	VOTE BLANC	1	-	VOTE NUL	1	-
Tintin	0	-	Gon Freecss	0	-	Mickey Mouse	0	-

Figure 5. Après avoir trier par le nombre de votes

Sur la Figure 5, un tri (toujours en affichage par ligne) selon le nombre de votes (c'est-à-dire de voix reçues sur l'ensemble des bulletins de vote) a été effectué. Il est possible de revenir à l'ordre initial. Il y a deux boutons différents pour les tris.

Passer à un autre vote | Trier par nombre de votes | Ordre Initial | passer en ligne

Nb voix = 25 (nb votants= 5.00)

Monkey D. Luffy	3	-
Natsu Dragnir	3	-
Astérix	3	-
Largo Winch	2	-
Eren Yeager	2	-
Rin Okumura	1	-
Lucky Luke	1	-
VOTE BLANC	1	-
VOTE NUL	1	-
Tintin	0	-
Gon Freecss	0	-
Mickey Mouse	0	-

Figure 6. Affichage en colonne

Un affichage en une colonne est illustré par la Figure 6. C'est utile notamment pour voir qui a reçu le plus de voix. Les égalités ne sont pas gérées : c'est l'ordre initial qui est conservé en cas d'égalité.

Le style présenté ici est épuré. Le design n'est pas l'objectif de cette ressource, il s'agit de la programmation. Si vous voulez consacrer quelques minutes au design, c'est votre choix, mais cela ne doit pas vous retarder dans la réalisation du sujet.



2 Prise en main de l'environnement, Création des fichiers et configuration initiale

2.1 Références sur les standards du Web

La référence en termes d'html, CSS et javascript se trouvent sur les sites suivants :

- <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (en anglais ou en français, il ne faut pas hésiter à regarder les deux langues, les pages ne sont pas identiques).
- <https://www.w3.org/> : sur ce site, vous pourrez trouver les spécifications officielles (mais pas facile à lire) des technologies du web. Par exemple : <https://html.spec.whatwg.org/multipage/> pour html, ou <https://www.w3.org/Style/CSS/Overview.en.html> et <https://www.w3.org/TR/css-2021/> pour CSS

D'autres sites (que vous trouverez bien vous-même) peuvent s'avérer utiles, attention à bien vérifier la date de l'information (les spécifications évoluent).

Pour le CSS (et même le reste), le site <https://caniuse.com/> peut vous aider à voir s'il faut des préfixes ou non pour les propriétés CSS, ou si tel ou tel « élément » est utilisable sur tel ou tel navigateur.

2.2 Edition de code html / css / javascript

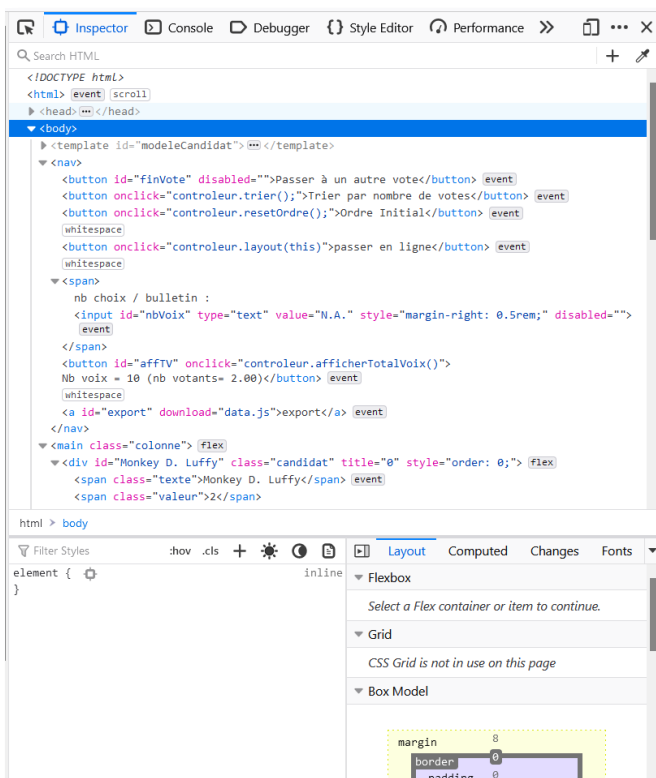


Figure 7. Les outils de développeur de Firefox

Pour le développement web, un simple éditeur de texte peut suffire. Cependant, la complétion ou la mise en valeur du code (syntax highlighting) sont des outils intéressants. Vous pouvez donc utiliser, selon ce que vous avez d'installer sur vos ordinateurs : notepad++ (éditeur très léger), VS Code (éditeur complet), un IDE comme Webstorm (la version pour js de IntelliJ) ou une version d'Eclipse pour le web. Peu importe, ce qui compte c'est que vous soyez à l'aise.

Il vous faut un navigateur, pour ouvrir les fichiers HTML en local (l'adresse commencera par : « file:/// »). Dans le navigateur, il faudra utiliser les outils de développement : via « inspecter » ou un raccourci (« F12 » sur windows).

Les outils de développement (c.f. Figure 7, la copie d'écran ci-contre, avec Firefox en anglais), comprend plusieurs onglets, dépendant du navigateur. Cependant nous retrouverons le plus souvent les outils suivants :

- L'inspecteur qui permet d'analyser l'arbre du document HTML et de voir les styles CSS. Vous pourrez y constater la structure du document, la manipuler et faire des essais avec CSS.
- La Console Javascript : quand vous utiliserez « console.log("coucou") ; », c'est là qu'il faudra vous rendre pour voir ce message... C'est également là que vous verrez **les erreurs javascript**. Vous pourrez aussi y essayer quelques lignes de javascript dans la console pour en constater les effets.
- Un debugger javascript, qui permet d'exécuter le code js en mode pas à pas et voir quand le programme ne se comporte pas comme prévu.



- Et d'autres que nous n'utiliserons pas dans le cadre de ce sujet : des outils de mesure de performance, du réseau, des vues sur d'autres dispositifs (comme un téléphone), etc.

2.3 Organisation de vos fichiers

Votre code javascript va être intégré à une page web. Pour des raisons de clarté d'organisation des fichiers, vous allez récupérer une archive (**début td3.zip dans moodle**) dont le code est organisé comme suit :

- Un fichier votes.html : cela sera le seul fichier html pour ce sujet. Pour exécuter votre TD/TP, il vous faudra ouvrir (ou rafraîchir) cette page.
- Un dossier src dans lequel il y aura :
 - Un dossier css pour y mettre vos feuilles de style (.css), il y a le fichier votes.css (déjà compléter).
 - Un dossier js pour y mettre vos fichier javascript (.js), il y a le fichier controleur.js (quasiment vide pour l'instant).
 - Dans le fichier html, il y a l'import du CSS et du javascript (balise <link> pour le CSS et balise <script> pour le javascript) les fichiers qui sont dans src. Quand vous devrez ajouter un fichier js, il faudra ajouter une balise <script> en conséquence. Notons que nous n'utiliserons pas les modules de javascript. Pour que votre code fonctionne, il faudra bien faire vos imports (et potentiellement l'ordre peut avoir un impact).

Il est demandé (car conseillé) de ne pas écrire du javascript exécuté au chargement mais de le déclencher quand la page est chargée. Comme nous n'utiliserons pas d'image pour ce sujet, dans les entêtes de votre page html, il y a déjà l'initialisation, dans la suite il faudra compléter la fonction init () :

```
<script type="text/javascript" >
  let controleur ;

  function init() {
    // c'est là que vous "lancerez" le code javascript : instantiation,
    // initialisation, abonnement aux événements, etc.
    controleur = new Controleur();
  }

  window.addEventListener("DOMContentLoaded", init);
</script>
```

Normalement, on n'expose pas des variables inutilement, mais vous pouvez pour cette fois, déclarer une variable globale « controleur » et l'instancier dans init. Cela permet d'y accéder dans la console, même si pour l'instant la classe Controleur est assez vide.

Un candidat aura la structure suivante :

```
<div class="candidat" id="unIdUnique">
  <span class="texte"></span><span class="valeur"></span>
  <button class="moins" disabled="disabled"></button>
</div>
```

Pour voir les différentes possibilités du css, il faut ajouter quelques div.candidat.vote (class="candidat vote") et modifier la classe CSS de <main> pour voir la mise en page « en ligne » (pas de classe css) ou « en une colonne » (class="colonne"). Pour faire vos essais, vous pouvez modifier votre code source et recharger la page ou modifier la page temporairement avec l'inspecteur (c.f. Figure 7).

Les explications sur le code html et CSS sont dans l'annexe 1 à la fin de ce sujet.



3 Premier pas en javascript : générer la liste des « candidats »

Plutôt que d'écrire en dure les « candidats », nous allons passer par un code de génération, en nous basant une liste (en fait un tableau) en javascript. Ce tableau contiendra des Candidats, des instances d'une classe Candidat que nous allons créer. Cette classe aura comme responsabilité de générer le code html correspondant, et plus tard, de gérer les événements. Pour générer le code html, nous nous baserons sur une <template>.

3.1 La balise template

Dans la balise <main>, à la place des <div> qui ont été écrites pour faire la maquette, nous allons mettre une balise <template> :

```
<template id="modeleCandidat"><div class="candidat">
  <span class="texte">à changer en js après import du template</span>
  <span class="valeur">0</span>
  <button class="moins" disabled="disabled">-</button></div>
</template>
```

C'est tout pour l'instant, mais en javascript, il sera possible de récupérer cette balise et d'en dupliquer le contenu.

3.1.1 La classe Candidat dans le fichier candidat.js et modification du Contrôleur

Dans cette classe nous allons définir un constructeur qui prendra en paramètre le nom du candidat et un contrôleur qui ne servira pas pour l'instant (on anticipe qu'il faudra compter les voix, désélectionner les candidats, etc.), deux méthodes : une pour générer le code html correspondant et un *getter* pour retourner ce code, et une méthode statique pour donner un tableau avec une liste de candidats. Le squelette de Candidat.js est donc :

```
class Candidat {
  static getListeInitiale(ctrl) {
    return [];
  }

  constructor(nom, ctrl) {
    this.nomCandidat = nom;
    this.controleur = ctrl;
  }

  genererVue() {
    console.log("pas encore implémenté");
    this.vue = null; // il faudra que cela soit un élément du DOM
  }

  getVue() {
    return this.vue;
  }
}
```

Pensez à ajouter la balise <script> dans votes.html pour importer le fichier « Candidat.js ».

Pour ne pas « coder sans voir l'effet du code », nous allons faire l'implémentation de « bout en bout » (là, c'est assez facile, nous restons côté client, dans le navigateur). Nous allons donc modifier en même temps le contrôleur et la classe Candidat et la fonction init() écrite dans la page « votes.html ». Voici les modifications à apporter :



- Dans la fonction `init()` de la page `votes.html`, après la création du contrôleur, il faut appeler la méthode `initialiser` sur ce contrôleur. C'est une nouvelle méthode qu'il faut créer dans la classe `Controleur`.
- Dans la classe `Controleur` :
 - Dans le constructeur, il faut définir un attribut « candidats » qui contient le résultat de l'appel à la méthode statique `Candidat.getListeInitiale` (en paramètre, il faut passer le contrôleur lui-même, c'est-à-dire `this`).
 - Dans la nouvelle méthode `initialiser()`, pour chaque candidat de l'attribut `candidats`, il faut générer la vue et ajouter la vue à la balise `<main>` présente dans la page web. Il s'agit là d'une première manipulation du DOM, pour cela utiliser : `document.querySelector` pour récupérer un objet représentant la balise `<main>` et sur cet objet utiliser `appendChild` avec le résultat de la méthode `getVue` d'un candidat.
- Dans la classe `Candidat` :
 - Implémentez la méthode statique `getListeInitiale`, pour qu'elle retourne un tableau de quelques candidats. Une façon de faire est :
 - De définir un tableau dans cette méthode, un tableau contenant des `String`, par exemple : « `let listeNomCandidats = ["Monkey D. Luffy", "Natsu Dragnir", "Astérix", "Tintin", "Largo Winch", "Rin Okumura", "Lucky Luke", "Eren Yeager", "Gon Freeecss", "Mickey Mouse", "VOTE BLANC", "VOTE NUL"];` »
 - Pour chacune de ses `String`, de créer un nouveau `Candidat` et de l'ajouter à un autre tableau
 - Ce tableau sera ce qui est retourné à la fin de cette méthode statique
 - Note : cette méthode s'apparente à une Fabrique (c'est un design pattern)
 - À ce stade, cela ne fonctionne pas encore car `getVue()` de `Candidat` retourne `null`. Dans la console javascript du navigateur, nous devrions observer « pas encore implémenté » puis une erreur (car `appendChild` ne fonctionne pas avec `null`). De fait, il va falloir utiliser le `<template>` dans la méthode `genererVue()`. C'est une autre manipulation du DOM. Il s'agit là de récupérer en javascript la balise `template` (`querySelector` est utilisable ici) et de récupérer le contenu de la `<template>` :

```
const template = document.querySelector("template");
this.vue = document.importNode(template.content, true).querySelector("div");
```

- Ensuite, il faut récupérer le `<span>` (`querySelector` en utilisant le sélecteur css « `span.texte` ») pour en changer le texte (grâce à `innerHTML`). Et c'est tout.

4 Interactivité : clic sur les « candidats » et changement de mise en page

Nous allons ajouter de l'interactivité en trois temps :

1. D'abord nous mettrons en place le fait de pouvoir cliquer sur un candidat (en deux étapes).
2. Puis « annuler la voix en cours de décompte » en cliquant sur le bouton. Pour l'instant, nous ne gérons pas le décompte global d'un bulletin (limitant à 5 voix).
3. Finalement nous pourrions changer de mise en page (avec le dernier bouton de la barre de contrôle).

4.1 Ajouter une voix sur un candidat

Dans la méthode `genererVue` de `candidat`, nous allons ajouter un écouteur d'événement « `click` » sur l'élément généré (grâce à la `<template>`) grâce à la méthode `addEventListener`. La question est de savoir ce qu'on écoute : la `<span>` qui correspond au nom du candidat, la `<div>` entière ? Vous pouvez expérimenter, mais nous conseillons de mettre sur la `<div>` pour ne pas à avoir à cliquer précisément sur une zone.



Le clic entraîne plusieurs conséquences :

- Mettre en valeur la div (via un changement de classe CSS, en utilisant toggle() de classList par exemple)
<https://developer.mozilla.org/fr/docs/Web/API/Element/classList>
- Augmenter le nombre de voix : il faut donc un attribut nbVoix à la classe Candidat, l'incrémenter, et changer le texte de la correspondant au nombre de voix.
- Activer le bouton « - » (c'est attribut disabled (un booléen) de l'objet javascript représentant le bouton).

À ce stade on peut enregistrer plusieurs votes d'un coup sur un candidat, la mise en forme est là une fois sur deux... nous allons corriger cela. De même, si vous cliquez sur le bouton « - » cela fait comme si vous cliquiez sur le div (ou rien si le bouton est désactivé). On va aussi corriger cela par la suite.

4.2 Limiter à un clic pris en compte (et afficher le +1 dans la valeur)

Pour éviter d'ajouter plusieurs voix sur un bulletin, par exemple par erreur avec un double clic impromptu, nous allons ajouter un état au candidat : « a reçu une voix » ou « n'a pas reçu une voix » :

- Si un candidat est dans l'état (initial) « n'a pas reçu une voix », alors il peut la recevoir et passer à l'état « a reçu une voix ». Nous en profiterons pour changer l'affichage. À la place d'avoir un affichage passer de 0 à 1, nous afficherons 0+1 comme sur la Figure 2. Notons que l'attribut nbVoix n'est plus incrémenté ici (cela serait fait plus tard quand nous pourrions passer à un autre bulletin).
- Si un candidat est dans l'état « a reçu une voix », nous allons ignorer le clic.
- Le retour à l'état « n'a pas reçu une voix » sera géré un peu plus tard.
- Cet état peut être modélisé et implémenté par un attribut (de type boolean, initialisé à false) aReçuUneVoix de la classe Candidat. Le traitement de l'événement commencera donc pas un test sur cet attribut. Dans le cas où le candidat a déjà reçu une fois, normalement il n'y a rien à faire, mais pour s'en rendre compte nous allons juste mettre un message dans la console, par exemple :

```
o console.log("voix déjà reçue, clic non pris en compte");
```

4.3 Retirer une voix sur un candidat

À ce stade, nous avons deux choses à régler avec le bouton moins : il n'est pas écouté (l'événement n'est pas pris en compte) et du coup, quand on clique dessus, avec la propagation de l'événement, c'est l'ajout d'une voix qui est ajoutée. Nous allons régler ceci.

Dans la méthode genererVue() de Candidat, ajouter un écouteur sur l'événement « click » sur le bouton. L'événement ne sera traité que si le candidat est dans l'état « a reçu une voix ». Dans ce cas, il faut revenir à l'état « n'a pas reçu une voix », recharger l'affichage du nombre de voix, ne plus mettre en valeur la <div> et désactiver le bouton. Comme cela fait plusieurs fois que l'affichage du nombre de voix est utilisé (à la génération, quand on enregistre une voix et maintenant quand on la retire), nous pouvons faire une méthode afficherNbVoix (appelée quand nécessaire) qui pourra traiter tous les cas : afficher le nombre de voix avec ou sans le « +1 ».

Dans tous les cas, l'événement « click » sur le bouton ne doit pas être propagé (méthode stopPropagation de l'événement).

4.4 Changer de mise en page

Cette fois, nous quittons (temporairement) la classe candidat pour aller dans la classe Controleur. Nous allons gérer le bouton « passer à une colonne ». Cela devrait être assez simple, si la maquette a bien été fait, il « suffit » de changer la classe CSS de la balise <main>. Nous mettrons aussi le texte du bouton en accord avec l'effet qu'il produira : « passer à une



colonne » ou « passer en ligne » (comme sur la Figure 6). L'abonnement à l'événement « click » sur ce bouton se fera dans la méthode initialiser de la classe Controleur. Rappelons que le bouton a un id « miseEnPage ». Notons aussi que la méthode toggle() de classList retourne un booléen : vrai si la classe CSS est ajoutée, faux si elle est supprimée. Cela permet de savoir si on ajoute ou pas la classe CSS « colonne » à <main>.

La facilité de ce changement de mise en page repose sur le CSS : plutôt que de tout faire en javascript, il est conseillé de déléguer un maximum au CSS et au HTML (même si cela n'est pas toujours possible).

5 Contrôle de la conformité d'un bulletin

Nous allons maintenant faire en sorte qu'un seul bulletin puisse être pris en compte à la fois. Nous allons faire cela en 3 étapes : pour 5 voix sur 5 candidats différents, la gestion des bulletins nuls ou blancs et finalement l'affichage du nombre total de voix.

5.1 5 voix

Pour l'instant, quand on clique sur un candidat, on affiche la prise en compte d'une voix. Seulement, un objet « Candidat » ne connaît pas les autres objets « Candidat », il ne sait pas combien d'autres candidats ont déjà été sélectionnés. Pour cela, l'objet « contrôleur » va permettre de centraliser les voix. En voici le principe :

1. Un candidat reçoit un clic et il est dans l'état « n'a pas reçu de voix ».
 - a. Si le candidat est dans l'état « a reçu une voix », le clic continue à être ignoré et le traitement s'arrête là.
2. Le candidat demande au contrôleur (qui a été passé en paramètre de son constructeur) s'il peut recevoir la voix (et valider le clic).
3. Le contrôleur vérifie (et décompte donc les voix).
 - a. S'il y a moins de 5 voix, c'est validé. Si c'est la 5^e voix, le bouton « passer à un autre vote » (c.f. Figure 3) est activé.
 - b. Sinon, ce n'est pas validé
4. Selon le retour (validé ou non validé), l'état du candidat change.

Si une voix est retirée (bouton « - » sur le candidat), il faut également prévenir le contrôleur pour qu'il modifie son décompte et qu'il désactive le bouton « passer à un autre vote » s'il y a moins de 5 voix.

Si le bouton « passer à un autre vote », chaque candidat met à jour son nombre de voix, tous ne sont plus sélectionnés et le bouton « passer à un autre vote » est désactivé.

Ce que vous avez à faire c'est donc :

- Interroger le contrôleur dans l'écouteur de l'événement « click » sur la <div> dans Candidat et selon le retour (true/false) du contrôleur, appliquer ou non la sélection (faites un console.log s'il n'y a rien à faire pour que vous puissiez le visualiser) (modification de l'écouteur concerné dans genererVue())
 - Créer une méthode ajouterUneVoix() dans Controleur qui retourne faux si l'ajout n'est pas possible (il y a déjà 5 voix) ou qui retourne vrai si c'est possible, incrémente alors nbVoixDuBulletin et si ce dernier atteint 5, il y a l'activation du bouton « passer à un autre vote »
- Gérer le décompte des voix dans Controleur



- Prévenir le contrôleur du retrait d'une voix quand on clique le bouton « - » d'un candidat (modification de l'écouteur concerné dans `genererVue()`). Pour cela faites une méthode `retirerUneVoix` dans la classe `Controleur` qui décrémente `nbVoixDuBulletin` (si `nbVoixDuBulletin > 0`) et qui désactive le bouton « passer à un autre vote »
- Faire une méthode `isChoisi()` dans `Candidat` qui retourne vrai si l'état est « a reçu une voix »
- Gérer le clic sur le bouton « passer à un autre vote » (ajout d'un écouteur dans `initialiser()` de `Controleur`)
 - Faire une méthode `validerChoix()` dans `Candidat` qui augmente son `nbVoix`, qui remet l'état à « n'a pas reçu de voix », qui actualise l'affichage du nombre de voix et qui retire la mise en valeur de la `<div>`.

5.2 Bulletins nuls ou blancs

Gérer les bulletins blancs (sans aucune réponse) ou nuls (avec moins de 5 noms de candidats, avec plus de 5, ou plusieurs fois le même), c'est géré un type de candidat « spécial », qui compte pour 5 voix. Pour cela nous allons ajouter une méthode `getValeurVoix()` qui retourne 1 dans `Candidat` et créer une classe `CandidatSpecial` qui hérite de `Candidat` (*extends*) et qui surcharge (uniquement) la méthode `getValeurVoix()` pour retourner 5. Il faudra alors modifier le tableau retourné par `Candidat.getListeInitiale(ctrl)` pour que les deux derniers éléments soient de type `CandidatSpecial`. Et il faut alors que le contrôleur utilise cette méthode pour incrémenter ou décrémente `nbVoixDuBulletin`, en faisant bien attention de ne pas dépasser 5 ou d'aller en négatif.

Il faudra ajouter un paramètre aux méthodes de `Controleur` (dans la classe `Candidat`) pour que l'objet contrôleur puisse savoir quel candidat l'appelle (ce candidat se passe lui-même en paramètre) ou alors le candidat passe la valeur de sa voix en paramètre (en appelant sa propre méthode `getValeurVoix()`).

Pensez aussi à mettre à jour des messages console (il ne concerne que les développeurs, mais un code à jour est toujours préférable).

5.3 Affichage du nombre total de voix

Il reste un dernier point à mettre à jour dans la barre de contrôle : le nombre de voix. À la validation de chaque bulletin, il faut recompter les voix et les afficher, en prenant bien en compte la valeur d'une voix (pour les « candidats spéciaux »). Dans la méthode `finBulletin` de `Controleur`, nous devons donc mettre à jour l'affichage de la balise `<span>` dont l'id est « affTV » (affichage total des voix). Pour cela, il faut parcourir la liste des candidats et faire la somme de leur voix en ajoutant un `getter` dans la classe `Candidat`. Ce `getter` fera lui-même la multiplication de `nbVoix` par `getValeurVoix()`.

6 Tri (graphiquement)

Il reste une dernière fonctionnalité non traitée : le tri. Les éléments html sont affichés généralement dans l'ordre dans lequel ils ont été ajouté au document. C'est le cas ici. Pour trier, on pourrait donc enlever toutes les `<div>` correspondantes aux candidats de la balise `<main>` (*removeChild*) et les rajouter dans le bon ordre (*appendChild* après un tri) à cette même balise. Pour remettre dans l'ordre initial, il faudrait conserver cet ordre-là et refaire la manipulation (*removeChild* / *appendChild*). À la place, ici, nous allons utiliser une propriété du `display flex`, la propriété `order` (à mettre aux `<div>`) :

<https://developer.mozilla.org/fr/docs/Web/CSS/order>

Nous allons ajouter deux méthodes à `Candidat` :

- Une méthode « ordonner » pour modifier cette propriété via `vue.style.order` en lui mettant en valeur un nombre négatif dont la valeur absolue est `nbVoix` (les éléments avec un « order » les plus petits sont affichés en premier).
- Une méthode « ordreInitial » pour remettre cette propriété à 0 (c'est la valeur par défaut): quand des éléments ont le même « order » ils sont affichés dans l'ordre d'ajout.



- Il ne vous reste plus qu'à faire deux écouteurs pour les deux boutons « trier par nombre de votes » (id « trier ») et « ordre initial » (id « ordreInitial ») dans la méthode initialiser de Controleur.



7 Annexe 1 : explication sur la maquette initiale

Dans cette section, il n'y a des explications sur la réalisation de la maquette en html / CSS. En partant des explications de point 1, une version « html /css » de la page (sans interaction, juste pour la présentation) a été réalisée, en voici les étapes. La structure de la page, illustrée par la Figure 1, est composée de deux zones :

- Une barre de contrôle, avec les boutons et l'information sur le nombre de voix, que l'on placera dans une balise <nav>
- La zone avec les « candidats » que l'on placera dans une balise <main>

La barre de contrôle ne pose pas de problème particulier : il s'agit de 4 boutons, d'un retour à ligne (
) et d'un texte (à placer dans un span pour pouvoir le manipuler plus tard dans le javascript). Afin d'anticiper les manipulations en javascript, il y a des id dans les balises : les 4 boutons auront pour id respectivement (dans l'ordre d'apparition) « finBulletin », « trier », « ordreInitial », « miseEnPage » ; le span aura pour id « affichageTotalVoix ». Le bouton « finBulletin » est inactif avec l'attribut disabled="disabled". La mise en forme des boutons est faite à l'aide des propriétés border-radius, padding, border, box-shadow, margin et la pseudo classe button:disabled

Un candidat aura la structure suivante :

```
<div class="candidat" id="unIdUnique">
  <span class="texte"></span><span class="valeur"></span>
  <button class="moins" disabled="disabled"></button>
</div>
```

Pour leur mise en forme de <main> et pour l'intérieur des <div class="candidat">, les flex-box ont été utilisées : <https://developer.mozilla.org/fr/docs/Web/CSS/flex> :

- Pour <main>, en affichage par ligne (sur plusieurs lignes), c'est assez simple : avec un « display : flex ; » et avec :
 - Une direction en ligne (« row », c'est-à-dire horizontale) <https://developer.mozilla.org/fr/docs/Web/CSS/flex-direction>
 - Une non-limitation à une ligne (« wrap ») <https://developer.mozilla.org/fr/docs/Web/CSS/flex-wrap>
 - Le nombre de div (c'est-à-dire de « candidats ») est déterminé par la taille des div, celle-ci se fixe avec la propriété flex-basis et selon le nombre variera. Par exemple avec 40rem il n'y aura que 2 div par ligne, avec 20rem il y en aura 4. Cela dépend aussi du niveau de zoom de la page. <https://developer.mozilla.org/fr/docs/Web/CSS/flex-basis>
- Pour <main>, en affichage par colonne, c'est également assez simple : toujours avec « display : flex ; » mais avec :
 - Une direction en colonne, et une limitation à une colonne (no-wrap)
 - Et cette fois-ci, flex-basis déterminera la hauteur (flex-basis détermine la longueur de la dimension dans le sens du flex : la largeur quand c'est en ligne, la hauteur quand c'est en colonne).
 - Pour différencier les deux cas, une classe css pour main (main.colonne par exemple) et le sélecteur « main.colonne > div.candidat » permettent d'écrire les règles pour les deux cas
- Les « div.candidat » seront aussi en flex, pour permettre d'avoir un texte qui prend de plus en plus de place (s'il y en a) et pour mettre le nombre de voix et le bouton à droite
 - « div.candidat » et « span.texte » peuvent donc occuper plus d'espace (flex-grow) <https://developer.mozilla.org/fr/docs/Web/CSS/flex-grow>

Développement d'applications avec IHM

TD n° 3



- « span.valeur » aura une marge à gauche automatique (auto) et une marge à droite pour qu'elle ne colle pas le bouton.

Il y a aussi :

- Le survol d'un candidat, comme l'illustre la Figure 8, doit permettre la mise en valeur du candidat : en changeant le curseur (ici grabbing), en empêchant la sélection de texte, et en changeant l'apparence de span.texte. Pour cela, les propriétés CSS suivantes ont été utilisées :
 - <https://developer.mozilla.org/fr/docs/Web/CSS/user-select>
 - <https://developer.mozilla.org/fr/docs/Web/CSS/cursor>
 - Faites bien attention aux sélecteurs (c'est quand on survole la div.candidat, pas quand on survole la span.texte)

Pour la mise en forme sur les bords, l'ombre, les coins arrondis, c'est comme avec les boutons.



Figure 8. Le survol d'un candidat avec la souris

Il est aussi prévu une class div.vote (« voté », mais on ne mettra pas l'accent dans le nom de la classe css) ou div.candidat.vote pour mettre en forme un candidat qui a reçu une voix pendant la saisie d'un bulletin en le modifiant ainsi :

```
div.candidat.vote {
  background: rgba(192,255,192,0.5);
  box-shadow: 0 0 0.4rem 0.2rem green;
}
```